EE2211 Introduction to Machine Learning

Lecture 4

Shaojing Fan
fanshaojing@nus.edu.sg

Office hour (hybrid): Friday 10.30-11.30am, E4-05-26 (Weeks 5-10)

© Copyright EE, NUS. All Rights Reserved.

Syllabus

- Introduction and Preliminaries (Yueming)
  - Introduction
  - Data Engineering
  - Introduction to Probability, Statistics, and Matrix
- Fundamental Machine Learning Algorithms I (Shaojing)
  - Systems of linear equations
  - Least squares, Linear regression
  - Ridge regression, Polynomial regression
- Fundamental Machine Learning Algorithms II (Shaojing)
  - Over-fitting, bias/variance trade-off
  - Optimization, Gradient descent
  - Decision Trees, Random Forest
- Performance and More Algorithms (Yueming)
  - Performance Issues
  - K-means Clustering
  - Neural Networks

Fundamental algorithms

Module II
(2 assignments, 20%)

Module III
(1 assignment, 16%)

© Copyright EE, NUS. All Rights Reserved.

Overview of Modules II & III

Lecs 4 & 5: Linear regression 📈 – our very first machine learning model!

Lec 6: Polynomial regression 🌈 – our second model, a more powerful extension of linear regression

Lecs 6-7: Key challenges in model training ⚠️, and strategies to build better models

Lec 8: Gradient descent 🚀 – the engine that powers model training and modern AI!

Lec 9: Decision Tree 🌳 – an intuitive and more explainable model

© Copyright EE, NUS. All Rights Reserved.

Fundamental ML Algorithms: Linear Regression

References for Lectures 4-6:

Main

- [Book1] Andriy Burkov, "**The Hundred-Page Machine Learning Book**", 2019. (read first, buy later: http://themlbook.com/wiki/doku.php)
- [Book2] Stauart Russell and Peter Norvig. **"Artificial Intelligence: A Modern Approach"**, Person, 2022

Supplementary

- [Book3] Stephen Boyd and Lieven Vandenberghe, "**Introduction to Applied Linear Algebra**", Cambridge University Press, 2018 (available online) http://vmls-book.stanford.edu/
- [Ref4] **Professor Vincent Tan's notes (chapters 4-6): (useful) https://vyftan.github.io/papers/ee2211book.pdf**

© Copyright EE, NUS. All Rights Reserved.

Assignment Arrangements

3 Assignments:

#	Release	Due	GAs in Charge
A1 (5%)	Week 4 Friday	Week 6 Friday	Chi Phan (chiphan@u.nus.edu); Le Viet Nguyen (nguyen.lv@u.nus.edu)
A2 (15%)	Week 6 Friday	Week 9 Friday	Yan Weilong (yanweilong@u.nus.edu); Guo Jingyuan (e0724521@u.nus.edu)
A3 (16%)	Week 9 Friday	Week 13 Friday	Mao Junyuan (junyuanm@u.nus.edu); Aditya Nawal (adityanawal@u.nus.edu)

- For any questions related to assignments, such as consultations or grading, please reach out to the **corresponding GAs in charge.**

- Python is required to complete the assignments. If you're not fully confident in your Python skills, feel free to consult your tutors and review the **two Python Clinic Sessions** (by Mr Yunze Leng) on Canvas. Additional five Python clinic sessions will be offered from Week 8 onwards by Mr. Nguyen Duc Thang.

- Late submission policy:

- Within 2 days: up to 80% of your grade (≤12 hrs late: 80%, 1 day late: 50%, 2 days late: 30%)
- After 2 days: 0 marks

© Copyright EE, NUS. All Rights Reserved.

Some Housekeeping

- **Office Hour (Hybrid)**: **Friday 10.30-11.30am (weeks 5-10).** Feel free to stop by my office (E4-05-26) or online anytime during office hours with questions.

- You're welcome to ask questions anytime during class. For online participants, please feel free to post your questions in the chat, and I will set aside dedicated time to read and address them. *As this is a large class, I appreciate your understanding if I'm not able to respond immediately.*

- If possible, I strongly encourage in-person attendance, as it allows for more interaction and a more effective learning experience.

- You can reach me through Canvas messaging or email at fanshaojing@nus.edu.sg. If you don't hear back within 48 hours (excluding weekends and public holidays), please kindly send a reminder.

© Copyright EE, NUS. All Rights Reserved.

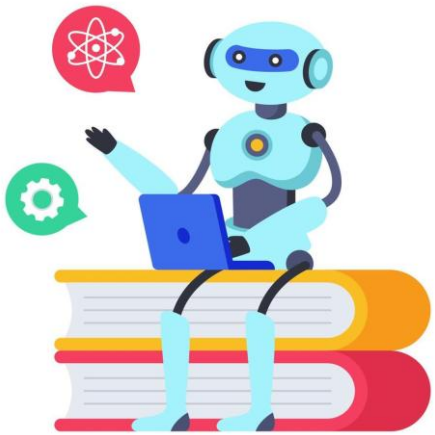
Acknowledgement

These course slides are prepared for teaching purposes and include adapted materials from online sources, as well as content developed by the EE2211 course development team, including Xinchao, Helen, Vincent, Thomas, Kar-Ann, Vincent, Chen Khong, Robby, and Haizhou.

© Copyright EE, NUS. All Rights Reserved.

When a machine "learns," what do you think it is actually doing? Answer in 1-3 words.

PollEv.com/shaojingfan569



If prompted to register on PollEV, simply click "Skip."

© Copyright EE, NUS. All Rights Reserved.

Syllabus

Module II Contents

- Operations on Vectors and Matrices
- Systems of Linear Equations
- Set and Functions
- Derivative and Gradient
- Least Squares, Linear Regression
- Linear Regression with Multiple Outputs
- Linear Regression for Classification
- Ridge Regression
- Polynomial Regression

Linear algebra is the "language" of machine learning

© Copyright EE, NUS. All Rights Reserved.

Agenda

- Motivation
- Operations on Vectors and Matrices
- Systems of Linear Equations

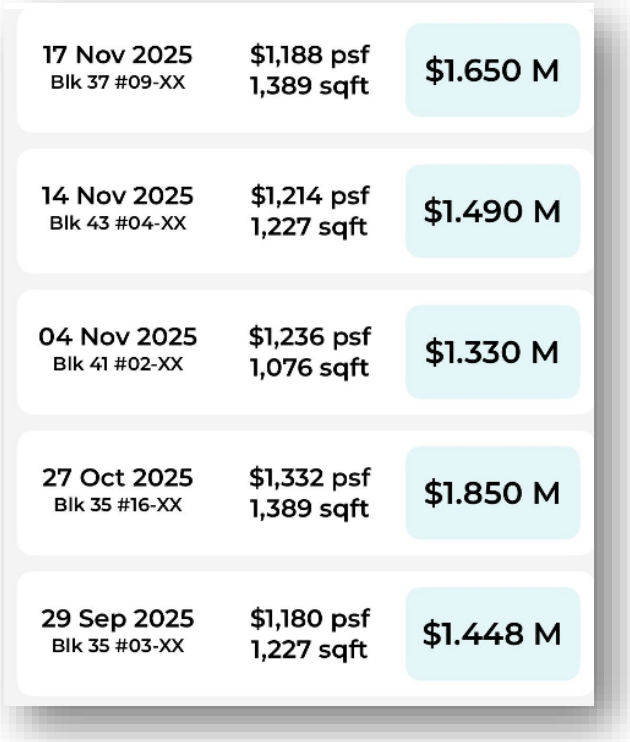
© Copyright EE, NUS. All Rights Reserved.

Linear Algebra & Machine Learning

Example: I want to sell my 1,400 ft² house—how much should I ask for?

Historical records:

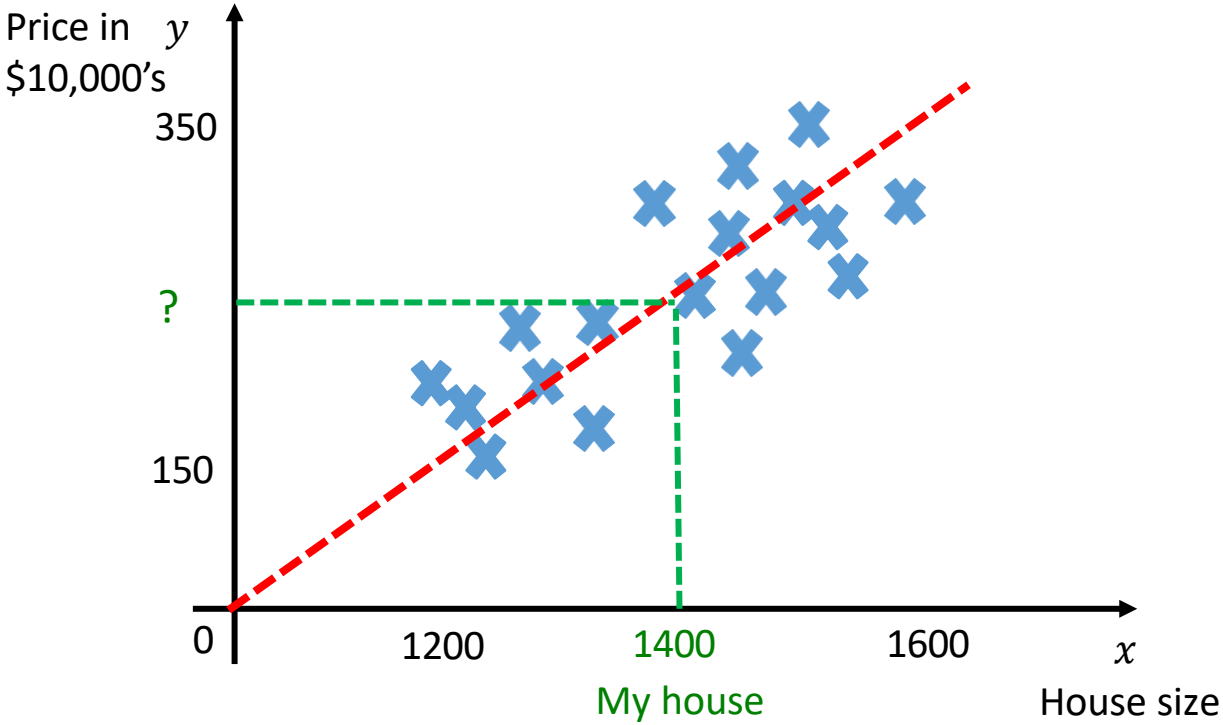
House size (feet^2)	Sold price ($10,000)
1389	165
1227	149
1076	133
1089	185
1227	145
…..	…..



© Copyright EE, NUS. All Rights Reserved.

Linear Algebra & Machine Learning

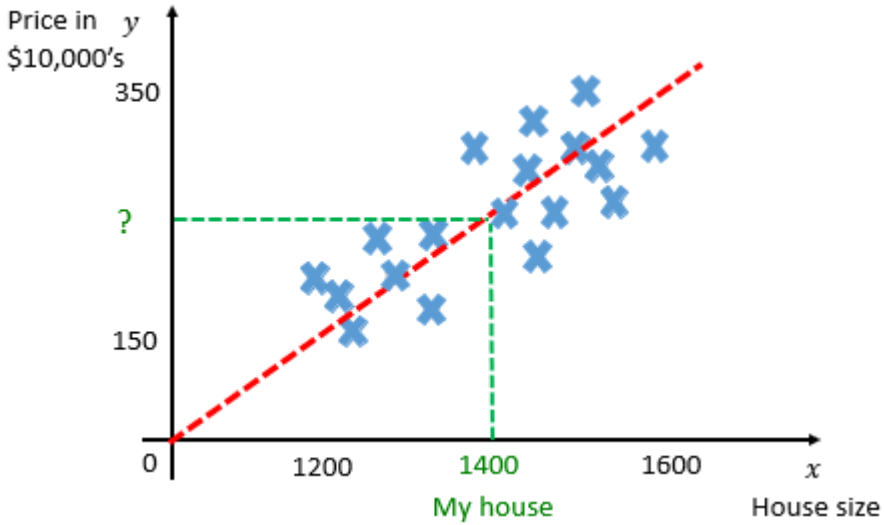
Example: I want to sell my 1,400 ft² house—how much should I ask for?



17 Nov 2025 Blk 37 #09-XX	$1,188 psf 1,389 sqft	$1.650 M
14 Nov 2025 Blk 43 #04-XX	$1,214 psf 1,227 sqft	$1.490 M
04 Nov 2025 Blk 41 #02-XX	$1,236 psf 1,076 sqft	$1.330 M
27 Oct 2025 Blk 35 #16-XX	$1,332 psf 1,389 sqft	$1.850 M
29 Sep 2025 Blk 35 #03-XX	$1,180 psf 1,227 sqft	$1.448 M

© Copyright EE, NUS. All Rights Reserved.

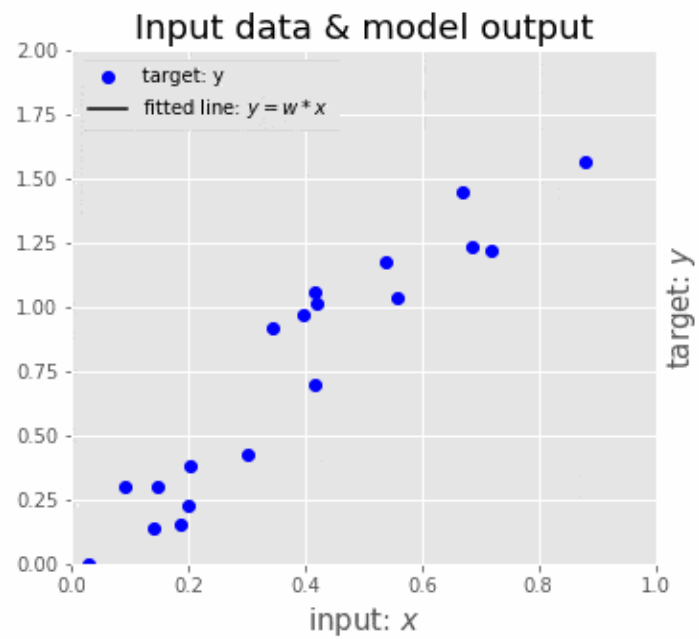
Linear Algebra & Machine Learning



A model with one feature x

$$f(x) = wx$$

Target: find w



Source: https://bharathikannann.github.io/blogs/a-visual-intro-to-linear-regression-math/

© Copyright EE, NUS. All Rights Reserved.

Linear Algebra & Machine Learning

A model with multiple features:

$$f(x) = w_1 x_1 + w_2 x_2 + \cdots w_d x_d$$

Size location … age

- Illustrative example:

House price = $0.8 \times house\ size + 0.4 \times location\ score + 0.2 \times number\ of\ bedrooms - 0.5 \times age\ of\ the\ house$

© Copyright EE, NUS. All Rights Reserved.

Linear Algebra & Machine Learning

A model with multiple features:

$$f(x) = w_1 x_1 + w_2 x_2 + \cdots w_d x_d$$

Suppose we have m house samples, each with d features and a selling price, and we want to learn unknown **weights** $w_1, \ldots, w_d$.

Size *location* … *house age* *sell price*

$$x_{11} w_1 + x_{12} w_2 + \cdots + x_{1d}\, w_d = y_1 \quad \textit{Sample 1}$$
$$x_{21} w_1 + x_{22} w_2 + \cdots + x_{2d}\, w_d = y_2 \quad \textit{Sample 2}$$
$$\vdots \qquad\qquad \vdots$$
$$x_{m1} w_1 + x_{m2} w_2 + \cdots + x_{md}\, w_d = y_m \quad \textit{Sample } m$$

System of linear equations

© Copyright EE, NUS. All Rights Reserved.

Linear Algebra & Machine Learning

$$
\begin{aligned}
x_{11}w_1 + x_{12}w_2 + \cdots + x_{1d}\, w_d &= y_1 \\
x_{21}w_1 + x_{22}w_2 + \cdots + x_{2d}\, w_d &= y_2 \\
&\vdots \\
x_{m1}w_1 + x_{m2}w_2 + \cdots + x_{md}\, w_d &= y_m
\end{aligned}
$$

In machine learning, we typically **use matrix notation:**

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

where:

feature 1 feature 2 … feature d

$$
\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{m3} \end{bmatrix}
\begin{array}{l} \textit{sample 1} \\ \textit{sample 2} \\ \vdots \\ \textit{sample } m \end{array}
$$

To be learned!

$$
\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}
\qquad
\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}
\begin{array}{l} \textit{price of house 1} \\ \vdots \\ \textit{price of house } m \end{array}
$$

© Copyright EE, NUS. All Rights Reserved.

Agenda

- Motivation
- Operations on Vectors and Matrices
- Systems of Linear Equations

© Copyright EE, NUS. All Rights Reserved.

Recap on Notations, Vectors, Matrices

Concept	Definition	Notation	Dimension	Key characteristics
Scalar	A single numerical value	A single number (e.g., 5, -2.3, π)	1 × 1	- A fundamental, indivisible quantity. - Has magnitude only.
Variable	A symbol representing a quantity that can change or take on different numerical values.	A letter (e.g., x, y, z, w_1)	1 × 1	- Represents a point in space, a direction, or a collection of features. - Has both magnitude and direction. - A 1D array of scalars.
Vector	An ordered list (array) of numbers (scalars).	Bold lowercase letters (e.g., $\mathbf{x}$, $\mathbf{y}$), or with an arrow ($\vec{\mathbf{x}}$); written as a row or column (e.g., $\mathbf{x}$ = [2, -1, 4])	n × 1 or 1 × n	- Represents a point in space, a direction, or a collection of features. - Has both magnitude and direction. - A 1D array of scalars.
Matrix	A rectangular array of numbers (scalars) arranged in rows and columns.	Bold uppercase letters (e.g., $\mathbf{A}$, $\mathbf{M}$), e.g., $\mathbf{X}$ = [[1, 2], [3, 4]]	m × n	- Organizes multi-dimensional data. - Can represent linear transformations or systems of equations. - A 2D array of scalars.

Capital Sigma: $\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_{n-1} + x_n$

Capital Pi: $\prod_{i=1}^{n} x_i = x_1 \cdot x_2 \cdot \ldots \cdot x_{n-1} \cdot x_n$

Reference: http://www.cs.cmu.edu/~zkolter/course/linalg

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Addition and subtraction of matrices and vectors is defined just as addition or subtraction of the elements, for $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$,

$$\mathbf{C} \in \mathbb{R}^{m \times n} = \mathbf{A} + \mathbf{B} \Leftrightarrow c_{ij} = a_{ij} + b_{ij}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \qquad \mathbf{x} + \mathbf{y} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ \vdots \\ x_n + y_n \end{bmatrix}$$

element-wise operation

- Example:

$$\begin{bmatrix} 2 & 8 \\ 3 & 7 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 0 & 5 \end{bmatrix} = \begin{bmatrix} 2+1 & 8+2 \\ 3+0 & 7+5 \end{bmatrix} = \begin{bmatrix} 3 & 10 \\ 3 & 12 \end{bmatrix}$$

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix or Vector Transpose:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad \mathbf{x}^T = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix}, \qquad \mathbf{X}^T = \begin{bmatrix} x_{11} & \cdots & x_{m1} \\ \vdots & \ddots & \vdots \\ x_{1n} & \cdots & x_{mn} \end{bmatrix}$$

- Example:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}, \qquad \mathbf{X}^T = \begin{bmatrix} x_{11} & x_{21} & x_{31} \\ x_{12} & x_{22} & x_{32} \\ x_{13} & x_{23} & x_{33} \end{bmatrix}$$

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Dot Product or Inner Product of Vectors:

$$\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^T \mathbf{y}$$
$$= [x_1 \quad x_2] \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$
$$= x_1 y_1 + x_2 y_2$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad \mathbf{x}^T = [x_1 \quad x_2]$$

Dimension must match

- Example:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

scalar

$$\mathbf{a} \cdot \mathbf{c} = 2*1 + 3*0 = \boxed{2}$$

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix-Vector Product

$$\mathbf{Xw} = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

Dimension must match

$$= \begin{bmatrix} x_{11}w_1 + x_{12}w_2 + x_{13}w_3 \\ x_{21}w_1 + x_{22}w_2 + x_{23}w_3 \end{bmatrix}$$

- Example:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{w} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$$\mathbf{X} \cdot \mathbf{w} = \begin{bmatrix} 1 \times 5 + 2 \times 6 \\ 3 \times 5 + 4 \times 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix-Vector Product (cont.): Visualization

$$\begin{pmatrix} a & b & c \\ l & m & n \\ r & s & t \end{pmatrix}\begin{pmatrix} A_x \\ A_y \\ A_z \end{pmatrix} \longrightarrow \begin{pmatrix} \quad \\ \quad \\ \quad \end{pmatrix}$$

Source:https://phys.libretexts.org/Courses/University_of_California_Davis/UCD%3A_Physics_9HB__Special_Relativity_and_Thermal_Statistical_Physics/3%3A_Spacetime/3.1%3A_Vector_Rotations

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix-Matrix Product: for two matrices $\mathbf{A} \in \mathbb{R}^{m \times d}, \mathbf{B} \in \mathbb{R}^{d \times h}$,

$$\mathbf{C} \in \mathbb{R}^{m \times h} = \mathbf{AB} \Leftrightarrow c_{ij} = \sum_{k=1}^{d} a_{ik} b_{kj}$$

Dimension must match

- Example:

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}, \quad B = \begin{bmatrix} j & k & l \\ m & n & o \\ p & q & r \end{bmatrix}$$

$$A \times B = \begin{bmatrix} aj + bm + cp & ak + bn + cq & al + bo + cr \\ dj + em + fp & dk + en + fq & dl + eo + fr \\ gj + hm + ip & gk + hn + iq & gl + ho + ir \end{bmatrix}$$

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix-Matrix Product: Visualization

$$\begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 3 & 1 & 5 \\ -2 & 1 & 3 \end{bmatrix} = \begin{bmatrix} & & \\ & & \end{bmatrix}$$

$$(2 \times 2) \qquad (2 \times 3) \qquad (2 \times 3)$$

Source: https://notesbylex.com/matrix-multiplication

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix inverse

Recall: $ab = c,\ a = c * b^{-1}$ $\frac{c}{b}$

The **identity matrix** $\mathbf{I} \in \mathbb{R}^{n \times n}$ is a ***square matrix*** with:

- 1s on the diagonal
- 0s elsewhere

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

It acts like **"1"** in matrix multiplication, *i.e.*, for any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$:

$\mathbf{IA} = \mathbf{A}$ (with $\mathbf{I} \in \mathbb{R}^{m \times m}$)

$\mathbf{AI} = \mathbf{A}$ (with $\mathbf{I} \in \mathbb{R}^{n \times n}$)

For a ***square matrix*** $\mathbf{A} \in \mathbb{R}^{n \times n}$, its **inverse matrix** $\mathbf{A}^{-1}$ is the matrix such that,

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I} = \mathbf{A}\mathbf{A}^{-1}$$

NOT all matrices have an inverse. Only square matrices with linearly independent rows (or columns) are invertible (nonsingular).

Ref: https://en.wikipedia.org/wiki/Invertible_matrix

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix inverse computation

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})}\operatorname{adj}(\mathbf{A})$$

$\mathbf{A}^{-1}$ only exists if det ($\mathbf{A}$) is ***not zero.***

- $\det(\mathbf{A})$ is the determinant of $\mathbf{A}$ **(a scalar, only exists for *square* matrices)**
- $\operatorname{adj}(\mathbf{A})$ is the adjugate or adjoint of $\mathbf{A}$ (a matrix)

For detailed calculation of determinant and adjugate, refer to:
https://math.libretexts.org/Bookshelves/Linear_Algebra/Linear_Algebra_with_Applications_(Nicholson)/03%3A_Determinants_and_Diagonalization/3.02%3A_Determinants_and_Matrix_Inverses
https://en.wikipedia.org/wiki/Determinant

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

```python
# EE2211 Lecture 4 Matrix inversion
import numpy as np
from numpy.linalg import det
from numpy.linalg import inv

# Define a square matrix A
A = np.array([[1, 2],
              [3, 4]])

# Determinant of A
det_A = det(A)

# Inverse of A
A_inv = np.linalg.inv(A)

# Adjoint of A: adj(A) = det(A) * A^{-1}
adj_A = det_A * A_inv

print("A =\n", A)
print("det(A) =", det_A)
print("adj(A) =\n", adj_A)
print("A_inv =\n", A_inv)
```

- Start by importing the necessary libraries.
- NumPy is a powerful and commonly used library for numerical and matrix computations in machine learning.
- **det:** computes det(A)
- **inv:** computes $\mathbf{A}^{-1}$

Define a matrix with 2 rows, 2 columns

Computes the determinant of a matrix

Directly computes the inverse of a matrix

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})}\operatorname{adj}(\mathbf{A})$$

```
A =
 [[1 2]
 [3 4]]
det(A) = -2.0000000000000004
adj(A) =
 [[ 4. -2.]
 [-3.  1.]]
A_inv =
 [[-2.   1. ]
 [ 1.5 -0.5]]
```

$\det(\mathbf{A})$ is a scaler, $\mathbf{A}^{-1}$ only exists if $\det(\mathbf{A})$ is ***not zero.***

Complete demo code is uploaded on canvas

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix inverse

Matrix $\mathbf{A}$ $(n \times n)$ is invertible if

1. $\det(\mathbf{A}) \neq 0$
2. Columns of $\mathbf{A}$ are linearly independent
3. Rows of $\mathbf{A}$ are linearly independent
4. $\mathbf{A}$ is of full rank …
5. $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ is invertible

(The above are equal statements)

The rank(**X**) corresponds to the maximal number of linearly independent columns/rows of **X.**

Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, "Introduction to Applied Linear Algebra", Cambridge University Press, 2018 (Chp11.2, P203)

© Copyright EE, NUS. All Rights Reserved.

Quick Question 1

True or False: A matrix $\mathbf{A}$ is invertible if its columns are linearly independent, which also implies that its rows are linearly independent.

PollEv.com/shaojingfan569

If prompted to register on PollEV, simply click "Skip."

© Copyright EE, NUS. All Rights Reserved.

Quick Question 1

True or False: A matrix $\mathbf{A}$ is invertible if its columns are linearly independent, which also implies that its rows are linearly independent.

This statement only holds if $\mathbf{A}$ is a *square matrix*.

Given a square matrix $\mathbf{A}$, if its columns are linearly independent, it has a non-zero determinant (i.e., $\det(\mathbf{A}) \neq 0$), the matrix $\mathbf{A}$ is invertible.

If $\mathbf{A}$ is a square matrix, then linearly independent columns also imply linearly independent rows, because for a square matrix, the column rank equals the row rank.

Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, “Introduction to Applied Linear Algebra”, Cambridge University Press, 2018 (Chp11.2, P203)
Strang, G. “Introduction to Linear Algebra (5th ed.)”. Wellesley-Cambridge Press, 2016 (Chp 2.5, P82-P83)

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix inverse in machine learning:

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

Quick question 2:
Given $\mathbf{X}$ and $\mathbf{y}$, how to compute $\mathbf{w}$?

where:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{m3} \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

m: no. of samples, *d*: no. of feature dimensions

© Copyright EE, NUS. All Rights Reserved.

Operations on Vectors and Matrices

Matrix inverse in machine learning:

$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m\times d}, \mathbf{w} \in \mathcal{R}^{d\times 1}, \mathbf{y} \in \mathcal{R}^{m\times 1}$ $\mathbf{w}$: to be learned

m: no. of samples, *d*: no. of feature dimensions

Case 1: If $\mathbf{X}$ is invertible, i.e., $\mathbf{X}^{-1}\mathbf{X} = \mathbf{I}$

Solution: Pre-multiply both sides by $\mathbf{X}^{-1}$:

$$\mathbf{X}^{-1}\mathbf{X}\,\mathbf{w} = \mathbf{X}^{-1}\mathbf{y}$$

$$\hat{\mathbf{w}} = \mathbf{X}^{-1}\mathbf{y}$$

This case is rare in practice because $\mathbf{X}$ must be square $(m = d)$ to be invertible, while we often have more samples than features $(m > d)$ in real-world tasks.

How to compute $\mathbf{w}$ if $\mathbf{X}$ is not invertible?

In ML, we write $\hat{\mathbf{w}}$ to represent the **estimated** or **learned** weights from data — it's a specific point in the space of $\mathbf{w}$

© Copyright EE, NUS. All Rights Reserved.

Agenda

- Motivation
- Operations on Vectors and Matrices
- Systems of Linear Equations

© Copyright EE, NUS. All Rights Reserved.

Systems of Linear Equations

- Consider a system of m linear equations with d variables or unknowns $w_1, \ldots, w_d$:

$$
\begin{aligned}
x_{11}w_1 + x_{12}w_2 + \cdots + x_{1d}\, w_d &= y_1 \\
x_{21}w_1 + x_{22}w_2 + \cdots + x_{2d}\, w_d &= y_2 \\
&\vdots \\
x_{m1}w_1 + x_{m2}w_2 + \cdots + x_{md}\, w_d &= y_m\,.
\end{aligned}
$$

Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, “Introduction to Applied Linear Algebra”, Cambridge University Press, 2018 (Chp8.3)

© Copyright EE, NUS. All Rights Reserved.

Systems of Linear Equations

These equations can be written compactly in matrix notation:

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

Where

label

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

m: no. of samples, *d*: no. of feature dimensions

Note:

- The data matrix $\mathbf{X} \in \mathcal{R}^{m \times d}$ and the target vector $\mathbf{y} \in \mathcal{R}^{m}$ are given
- The unknown vector of parameters $\mathbf{w} \in \mathcal{R}^{d}$ is to be learnt

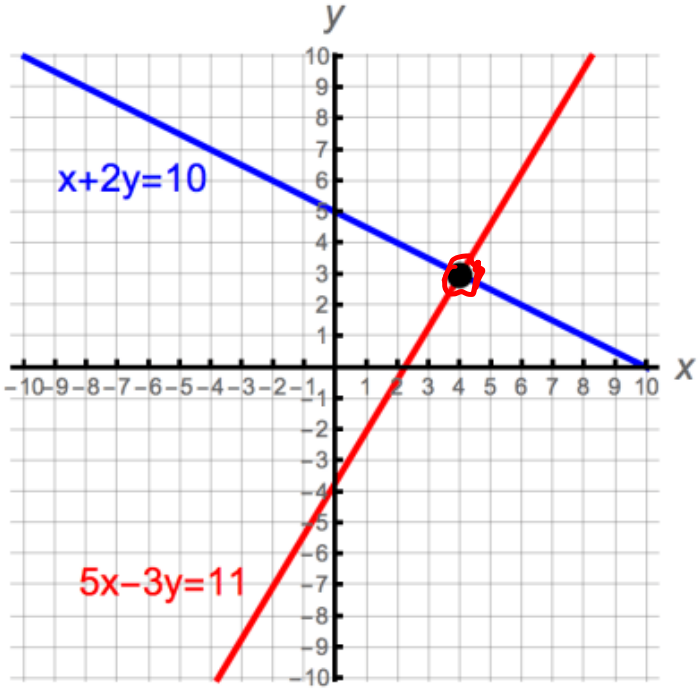
➢ Illustrative example:

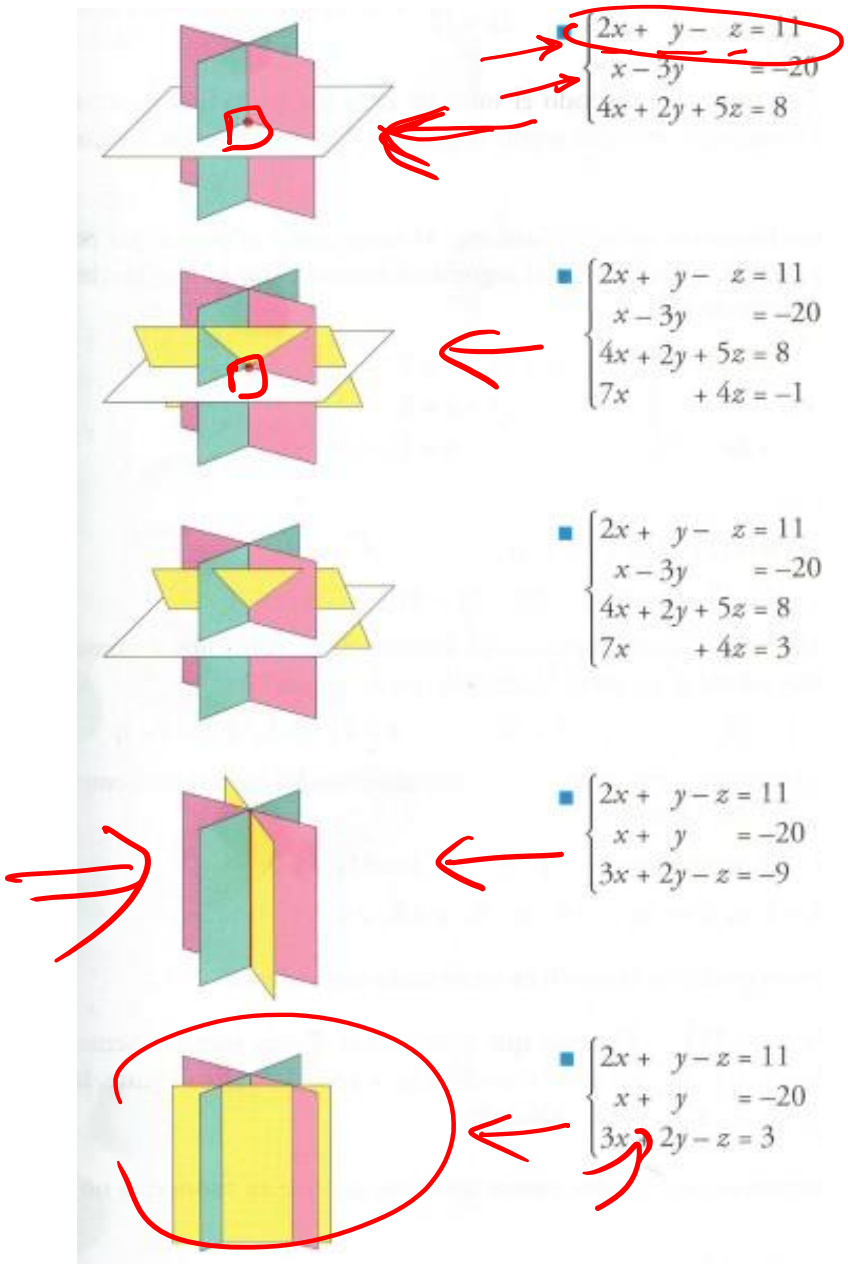
House price $= w_1 \times \textit{house size} + w_2 \times \textit{location score} + w_3 \times \textit{number of bedrooms} + w_4 \times \textit{age of the house}$

© Copyright EE, NUS. All Rights Reserved.

Systems of Linear Equations

Take 3D space as an example: Two points determine a line; three non-collinear points determine a plane.





Source: https://tex.stackexchange.com/questions/691744/types-of-systems-of-linear-equations-and-intersection-of-planes-with-tikz

© Copyright EE, NUS. All Rights Reserved.

Solving Systems of Linear Equations

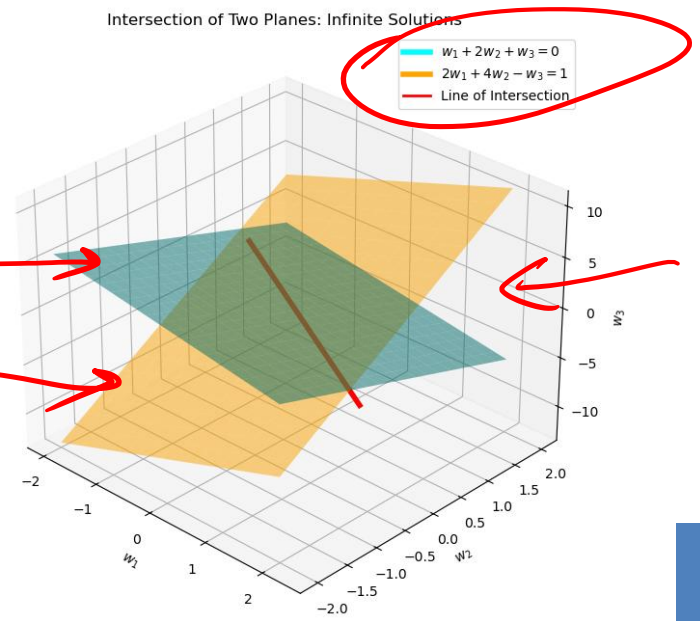
A set of linear equations can have no solution, one solution, or infinitely many solutions.

- Example:

Given $\mathbf{X}\mathbf{w} = \mathbf{y}$, where $\mathbf{X} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & -1 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

$$\begin{cases} w_1 + 2w_2 + w_3 = 0 \\ 2w_1 + 4w_2 - w_3 = 1 \end{cases}$$



Visualization in Python

© Copyright EE, NUS. All Rights Reserved.

Solving Systems of Linear Equations

Qn: Give a linear equation system (L.E.S), how can we tell if it's solvable, and whether the solution is unique or infinite?

- Consider a system of m linear equations with d variables or unknowns $w_1, \dots, w_d$:

$$
\begin{aligned}
x_{1,1}w_1 + x_{1,2}w_2 + \cdots + x_{1,d}\, w_d &= y_1 \\
x_{2,1}w_1 + x_{2,2}w_2 + \cdots + x_{2,d}\, w_d &= y_2 \\
&\vdots \\
x_{m,1}w_1 + x_{m,2}w_2 + \cdots + x_{m,d}\, w_d &= y_m\,.
\end{aligned}
$$

- The data matrix $\mathbf{X} \in \mathcal{R}^{m \times d}$ and the target vector $\mathbf{y} \in \mathcal{R}^{m}$ are given
- The unknown vector of parameters $\mathbf{w} \in \mathcal{R}^{d}$ is to be learnt.

m: no. of samples, *d*: no. of feature dimensions

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

Where

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \cdots & x_{m,d} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

To solve this, we first need to understand three key concepts (see the next slides).

© Copyright EE, NUS. All Rights Reserved.

Solving Systems of Linear Equations

Concept 1: Types of linear systems — even determined, over-determined, and under-determined:

$Xw = Y$

$\mathbf{X}$ is Square	Even-determined	m = d	Equal number of equations and unknowns
$\mathbf{X}$ is Tall	Over-determined	m > d	More number of equations than unknowns
$\mathbf{X}$ is Wide	Under-determined	m < d	Fewer number of equations than unknowns

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

Where

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \cdots & x_{m,d} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

m: no. of samples, *d*: no. of feature dimensions

© Copyright EE, NUS. All Rights Reserved.

Class Discussion

For each case, discuss: Is the system likely even-determined, over-determined, or under-determined? Why do you think so?

1. House Prices Prediction: Predicting house prices from a few factors like square footage, number of bedrooms, and age, and you have data from hundreds or thousands of houses sales.

2. Medical Diagnosis: Predicting patient outcomes using extensive lab tests, CT scans, and over 10,000 gene expression features for a rare disease, with only a limited number of patient records available.

3. Movie Ratings: Predicting the popularity of a new movie based on a few features—such as genre, director, and lead actor—using ratings from thousands of users across hundreds of movies.

PollEv.com/shaojingfan569

$\mathbf{X}$ is Square	Even-determined	m = d	Equal number of equations and unknowns
$\mathbf{X}$ is Tall	Over-determined	m > d	More number of equations than unknowns
$\mathbf{X}$ is Wide	Under-determined	m < d	Fewer number of equations than unknowns

© Copyright EE, NUS. All Rights Reserved.

Class Discussion

1. House Prices Prediction: Predicting house prices from a few factors like square footage, number of bedrooms, and age, and you have data from hundreds or thousands of houses sales. **Over-determined**

2. Medical Diagnosis: Predicting patient outcomes using extensive lab tests, CT scans, and over 10,000 gene expression features for a rare disease, with only a limited number of patient records available. **Under-determined**

3. Movie Ratings: Predicting the popularity of a new movie based on a few features—such as genre, director, and lead actor—using ratings from thousands of users across hundreds of movies. **Over-determined**

$\mathbf{X}$ is Square	Even-determined	m = d	Equal number of equations and unknowns
$\mathbf{X}$ is Tall	Over-determined	m > d	More number of equations than unknowns
$\mathbf{X}$ is Wide	Under-determined	m < d	Fewer number of equations than unknowns

m: no. of samples,
d: no. of feature dimensions

© Copyright EE, NUS. All Rights Reserved.

Class Discussion: Takeaway

System Type	When It Occurs	Real-World Examples
Even-determined ($m = d$)	Rare; exact match of samples and features	Small controlled experiments, some physics models
Over-determined ($m > d$)	Common in traditional statistics & traditional ML	Customer analytics (100K customers × 50 features), insurance pricing
Under-determined ($m < d$)	Two main scenarios: 1. High-dimensional data (images, text, genomics), increasingly popular in modern ML 2. Data scarcity (rare diseases, personalized medicine)	1. Medical imaging, NLP, genomics 2. Rare disease prediction, specialized healthcare

m: no. of samples, *d*: no. of feature dimensions

© Copyright EE, NUS. All Rights Reserved.

Solving Systems of Linear Equations

Concept 2: **Augmented matrix ($\widetilde{\mathbf{X}}$):** In our context, the augmented matrix plays a similar role to concatenating inputs and outputs

- Example:

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

$$\mathbf{X} = \begin{bmatrix} 95 & 86 \\ 80 & 75 \\ 70 & 72 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 86.9 \\ 75.5 \\ 71.8 \end{bmatrix}$$, **w** is to be solved

$$\widetilde{\mathbf{X}} = [\mathbf{X}\ \mathbf{y}] \in \mathcal{R}^{m\times(d+1)} = \begin{bmatrix} 95 & 86 & 86.9 \\ 80 & 75 & 75.5 \\ 70 & 72 & 71.8 \end{bmatrix}$$

Ref: https://maths.nuigalway.ie/~rquinlan/MA203/section1-5.pdf

© Copyright EE, NUS. All Rights Reserved.

Solving Systems of Linear Equations

Concept 3: Left and right inverse of X

$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m\times d}, \mathbf{w} \in \mathcal{R}^{d\times 1}, \mathbf{y} \in \mathcal{R}^{m\times 1}$ w: to be learned

m: no. of samples, *d*: no. of feature dimensions

Recall case 1: If $\mathbf{X}$ is invertible, i.e., $\mathbf{X}^{-1}\mathbf{X} = \mathbf{I}$

Solution: Pre-multiply both sides by $\mathbf{X}^{-1}$: $\mathbf{X}^{-1}\mathbf{X}\,\mathbf{w} = \mathbf{X}^{-1}\mathbf{y}$

$$\hat{\mathbf{w}} = \mathbf{X}^{-1}\mathbf{y}$$

How to compute $\mathbf{w}$ if $\mathbf{X}$ is not invertible?

We can use **pseudoinverse**!

$$\mathbf{X}\mathbf{w} = \mathbf{y} \quad \Rightarrow \quad \underbrace{\mathbf{X}^{\dagger}\mathbf{X}}_{\mathbf{I}}\mathbf{w} = \mathbf{X}^{\dagger}\mathbf{y} \quad \Rightarrow \quad \hat{\mathbf{w}} = \mathbf{X}^{\dagger}\mathbf{y}$$

A matrix **B** that satisfies $\boldsymbol{B}_{d\,x\,m}\boldsymbol{A}_{m\,x\,d} = \mathbf{I}$ is called a **left-inverse** of $\mathbf{A}$. The **left-inverse** of $\mathbf{X}$: $\mathbf{X}^{\dagger} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ given $\mathbf{X}^T\mathbf{X}$ is invertible.

$(\mathbf{X}^{\dagger}\mathbf{X} = \mathbf{I})$

$(m \geq d)$

A matrix **B** that satisfies $\mathbf{A}_{m\,x\,d}\mathbf{B}_{d\,x\,m} = \mathbf{I}$ is called a **right-inverse** of $\mathbf{A}$. The **right-inverse** of $\mathbf{X}$: $\mathbf{X}^{\dagger} = \mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}$ given $\mathbf{X}\mathbf{X}^T$ is invertible.

$(\mathbf{X}\mathbf{X}^{\dagger} = \mathbf{I})$

$(m \leq d)$

Ref: Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.), Section 2.5, P86

© Copyright EE, NUS. All Rights Reserved.

The Rouché–Capelli (RC) Theorem

$Xw = y$

Form the augmented matrix ($\widetilde{\mathbf{X}}$)

$$\widetilde{\mathbf{X}} = [\mathbf{X}\ \mathbf{y}] = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} & y_1 \\ x_{21} & x_{22} & \cdots & x_{2d} & y_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{m3} & y_m \end{bmatrix}$$

(i) The system $\mathbf{Xw} = \mathbf{y}$ has a unique solution if and only if

$$\text{rank}(\mathbf{X}) = \text{rank}(\widetilde{\mathbf{X}}) = d;$$

(ii) The system $\mathbf{Xw} = \mathbf{y}$ has no solution if and only if

$$\text{rank}(\mathbf{X}) < \text{rank}(\widetilde{\mathbf{X}});$$

(iii) The system $\mathbf{Xw} = \mathbf{y}$ has infinitely many solutions if and only if

$$\text{rank}(\mathbf{X}) = \text{rank}(\widetilde{\mathbf{X}}) < d.$$

d: number of feature columns in $\mathbf{X}$

The rank of a matrix $\mathbf{X}$ is the number of linearly independent rows or columns or $\mathbf{X}$.

https://vyftan.github.io/papers/ee2211book.pdf P24

© Copyright EE, NUS. All Rights Reserved.

Solutions of L.E.S (Based on RC Theorem)

For $\mathbf{X} \in \mathcal{R}^{m \times d}$ and $\widetilde{\mathbf{X}} = [\mathbf{X}\ \mathbf{y}] \in \mathcal{R}^{m \times (d+1)}$, the system $\mathbf{X}\mathbf{w} = \mathbf{y}$

	Even (m = d)	Over (m > d)	Under (m < d)
i) $\text{rank}(\mathbf{X}) = \text{rank}(\widetilde{\mathbf{X}}) = d$	Unique sol: $\hat{\mathbf{w}} = \mathbf{X}^{-1}\mathbf{y}$	Unique sol: $\hat{\mathbf{w}} = (\mathbf{X}^{\mathbf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathbf{T}}\mathbf{y}$	--
ii) $\text{rank}(\mathbf{X}) < \text{rank}(\widetilde{\mathbf{X}})$	No sol	No sol in general, but when left inv $\mathbf{X}$ exists, least-squares sol (approx.): $\hat{\mathbf{w}} = (\mathbf{X}^{\mathbf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathbf{T}}\mathbf{y}$	No sol
iii) $\text{rank}(\mathbf{X}) = \text{rank}(\widetilde{\mathbf{X}}) < d$	Infinitely many sol*	Infinitely many sol*	Infinitely many sol in general, but when right inv $\mathbf{X}$ exists, least-norm sol: $\hat{\mathbf{w}} = \mathbf{X}^{\mathbf{T}}(\mathbf{X}\mathbf{X}^{\mathbf{T}})^{-1}\mathbf{y}$

*These particular cases are not covered in this course.

In ML, we write $\hat{\mathbf{w}}$ to represent the **estimated** or **learned** weights from data — it's a specific point in the space of $\mathbf{w}$

© Copyright EE, NUS. All Rights Reserved.

Example 1

$$w_1 + w_2 = 1 \quad (1)$$
$$w_1 - w_2 = 0 \quad (2)$$
$$w_1 = 2 \quad (3)$$

Given the system of linear equation, compute the **least-squares solution** vector $\hat{\mathbf{w}}$

$$\underset{\mathbf{X}}{\begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 1 & 0 \end{bmatrix}} \underset{\mathbf{w}}{\begin{bmatrix} w_1 \\ w_2 \end{bmatrix}} = \underset{\mathbf{y}}{\begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}}$$

$Xw = Y$

$m = 3$

$d = 2$

- This corresponds to case (ii) of the RC Theorem for an over-determined system, where an exact solution generally does not exist. The least-squares solution provides the best approximate solution.

- Since $\mathbf{X}^{\mathbf{T}}\mathbf{X}$ is invertible (i.e., the left inverse exists), we can compute a least-squares solution (approx.):

$$\hat{\mathbf{w}} = (\mathbf{X}^{\mathbf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathbf{T}}\mathbf{y}$$

© Copyright EE, NUS. All Rights Reserved.

Example 1 (cont.)

$$\begin{array}{ccc} \mathbf{X} & \mathbf{w} & \mathbf{y} \end{array}$$

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}$$

L. E. S: $\mathbf{Xw} = \mathbf{y}$

$$\hat{\mathbf{w}} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y}$$

$$= \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$$

$X\underline{w} = Y$

$X\underline{w} \approx Y$

$\|Xw - Y\|^2 \to 0$

- This is the usual case for over-determined systems.
- No exact solution, and we look for an approximate solution, known as least square solution.

Idea: To find a solution $\hat{\mathbf{w}}$ that makes $\mathbf{X}\hat{\mathbf{w}}$ as close as possible to $\mathbf{y}$. We measure the closeness using squared error: $\| \mathbf{X}\hat{\mathbf{w}} - \mathbf{y} \|^2 = \sum_i (\underbrace{\mathbf{x}_i^{\mathrm{T}}\hat{\mathbf{w}}}_{\hat{y}_i} - y_i)^2$

© Copyright EE, NUS. All Rights Reserved.

Example 1 (cont.)

$$w_1 + w_2 = 1 \quad (1)$$
$$w_1 - w_2 = 0 \quad (2)$$
$$w_1 = 2 \quad (3)$$

$$\begin{array}{ccc} \mathbf{X} & \mathbf{w} & \mathbf{y} \\ \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 1 & 0 \end{bmatrix} & \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = & \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} \end{array}$$

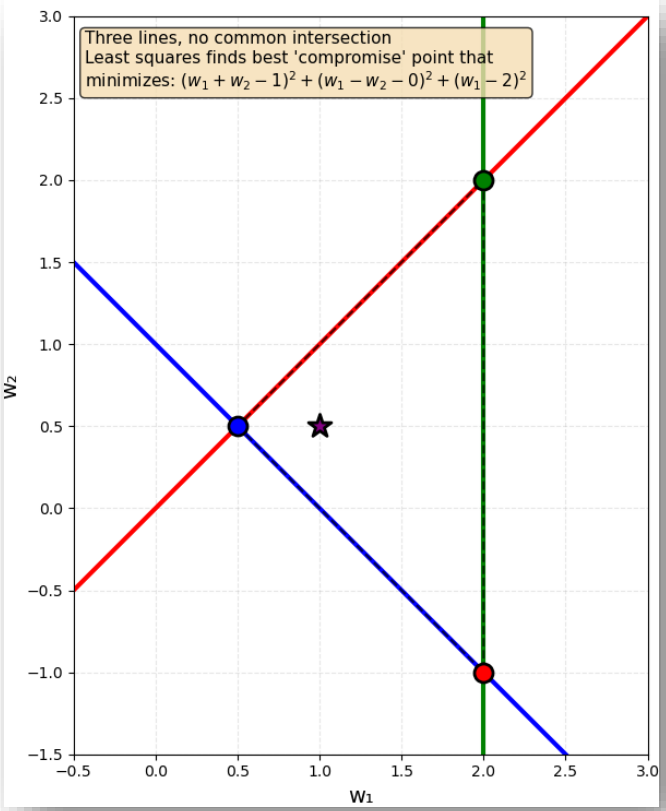
```python
# EE2211 Lecture 4 Over-determined system (m > d) Demo
import numpy as np
from numpy.linalg import inv

X = np.array([[1, 1],
              [1, -1],
              [1, 0]])
y = np.array([1, 0, 2])

# LEAST SQUARES SOLUTION
w = inv(X.T @ X) @ X.T @ y
print(w)
```

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Code	Shape	Type
y = np.array([1, 0, 2])	(3,)	1D array (agnostic)
y = np.array([[1], [0], [2]])	(3, 1)	Column vector (2D)
y = np.array([[1, 0, 2]])	(1, 3)	Row vector (2D)



Python demo

© Copyright EE, NUS. All Rights Reserved.

Example 2

$$w_1 + 2w_2 + 3w_3 = 2 \quad (1)$$
$$w_1 - 2w_2 + 3w_3 = 1 \quad (2)$$

Given the system of linear equation, compute the **least-norm solution** vector $\hat{\mathbf{w}}$

$$\begin{array}{ccc} \mathbf{X} & \mathbf{w} & \mathbf{y} \\ \begin{bmatrix} 1 & 2 & 3 \\ 1 & -2 & 3 \end{bmatrix} & \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = & \begin{bmatrix} 2 \\ 1 \end{bmatrix} \end{array}$$

$Xw = Y$

$m = 2, \quad d = 3$

- This corresponds to case (iii) of the RC Theorem for an under-determined system, where there are generally infinitely many solutions. The least-norm solution provides a unique solution with the one with the smallest Euclidean norm.

- Since $\mathbf{X}\mathbf{X}^{\mathbf{T}}$ is invertible (i.e., the right inverse exists), we can compute a least norm solution (constrained):

$$\hat{\mathbf{w}} = \boxed{\mathbf{X}^{\mathbf{T}}(\mathbf{X}\mathbf{X}^{\mathbf{T}})^{-1}}\mathbf{y}$$

© Copyright EE, NUS. All Rights Reserved.

Example 2 (cont.)

$$w_1 + 2w_2 + 3w_3 = 2 \quad (1)$$
$$w_1 - 2w_2 + 3w_3 = 1 \quad (2)$$

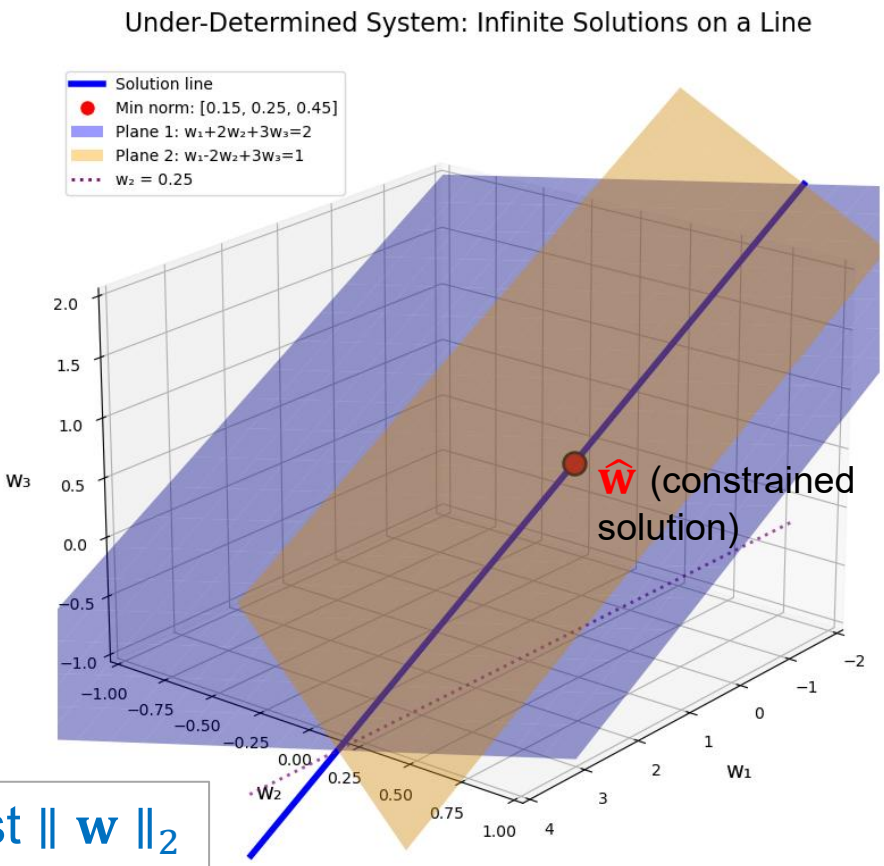
L. E. S: $\mathbf{X}\mathbf{w} = \mathbf{y}$

$$\begin{array}{ccc} \mathbf{X} & \mathbf{w} & \mathbf{y} \end{array}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 1 & -2 & 3 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\hat{\mathbf{w}} = \mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{y}$$

$$= \begin{bmatrix} 1 & 1 \\ 2 & -2 \\ 3 & 3 \end{bmatrix} \begin{bmatrix} 14 & 6 \\ 6 & 14 \end{bmatrix}^{-1} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.25 \\ 0.45 \end{bmatrix}$$



Constraint: The solution with the smallest $\| \mathbf{w} \|_2 = \sqrt{w_1^2 + w_2^2 + \cdots + w_d^2}$ among all solutions

© Copyright EE, NUS. All Rights Reserved.

Example 2 (cont.)

$$w_1 + 2w_2 + 3w_3 = 2 \quad (1)$$
$$w_1 - 2w_2 + 3w_3 = 1 \quad (2)$$

$$\begin{array}{ccc} \mathbf{X} & \mathbf{w} & \mathbf{y} \\ \begin{bmatrix} 1 & 2 & 3 \\ 1 & -2 & 3 \end{bmatrix} & \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} & = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \end{array}$$

```python
# EE2211 Lecture 4 Under-determined system (m < d) Demo
import numpy as np
from numpy.linalg import inv

# System: X * w = y
X = np.array([[1, 2, 3],
              [1, -2, 3]])
y = np.array([2, 1])

# LEAST-NORM SOLUTION
w = X.T @ inv(X @ X.T) @ y
print(w)
```

$$\hat{\mathbf{w}} = \mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{y}$$

Various methods for matrix multiplication:
-For 1D arrays: Use np.dot() or @ for dot products. Avoid np.matmul().
-For 2D arrays: np.dot(), @, and np.matmul() work the same.
-For 3D+ arrays: Use @ or np.matmul() (not np.dot()).

Python demo

© Copyright EE, NUS. All Rights Reserved.

Summary

Systems of Linear Equations $\mathbf{X}\mathbf{w} = \mathbf{y}$

- Matrix-vector notation, matrix inversion
- Even-, over-, under-determined linear systems

$\mathbf{X}$ is Square	Even-determined	m = d	One unique solution in general	$\hat{\mathbf{w}} = \mathbf{X}^{-1}\mathbf{y}$
$\mathbf{X}$ is Tall	Over-determined	m > d	No exact solution in general; An approximated solution	$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ Left-inverse
$\mathbf{X}$ is Wide	Under-determined	m < d	Infinite number of solutions in general; Unique constrained solution	$\hat{\mathbf{w}} = \mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{y}$ Right-inverse

In classical linear algebra, we check ranks first to see if a solution exists. In machine learning, however, we usually don't need to do this explicitly. Computing the pseudoinverse (left or right) automatically handles rank and finds a solution or best approximation, making explicit RC checks unnecessary in practice.

python package numpy
Inverse: numpy.linalg.inv(X)
Transpose: X.T

© Copyright EE, NUS. All Rights Reserved.

Announcement

- **Assignment 1 (5%)** will be released today, and due on Week 6 Friday. GAs in charge of Assignment 1:
  - Chi Phan (chiphan@u.nus.edu),
  - Le Viet Nguyen (nguyen.lv@u.nus.edu)

- **Tutorials in week 6 (CNY public holiday)**

  Please check with your respective tutor for any schedule arrangements.

- **Mid-term exam (30%)**
  - Covers L1 to L6 (Modules I & II)
  - Date & Time: Saturday, 14 March 2026, 4–5 pm
  - Venue: MPSH 2A & 2B
  - Note: Some questions from Module II will require programs to solve.

© Copyright EE, NUS. All Rights Reserved.

© Copyright EE, NUS. All Rights Reserved.

© Copyright National University of Singapore. All Rights Reserved.